

Week 21 | Autonomous prediction: representation, phase and dynamics

A forecast must live with its own history

A good one-step prediction is evaluated with a correct observed context. A deployed recurrent forecast is evaluated after that context contains model outputs. Week 18 introduced this difference through teacher forcing; here the consequence is measured on a physical wake trajectory.

The primary Re110 protocol fits a rank-eight vorticity representation on frames 0:160. Checkpoint selection scores an autonomous rollout through 160:210. Retained evaluation uses 210:281. The rollout starts only once from observed frames 156:160, and no true state is inserted at frame 210. Python slices are half-open throughout these notes.

Match the information given to baselines

The causal Transformer receives four past coefficient vectors and predicts all shifted positions during training. Its deployment output is the final predicted next state. The matched History-MLP receives the same four-frame context and predicts the next state. Its 250 hidden units give 10258 parameters, compared with 10256 in the Transformer. DMD uses the latest coefficient vector and a fitted linear propagator. Persistence repeats the last observed full field; the POD oracle projects the truth and is not deployable.

The original Week 7 MLP result is retained as a contextual reference, because omitting an already-existing strong baseline would misrepresent the course. The new matched MLP is the direct comparison: it uses the same data boundary, selected objective and fitting ceiling as the new Transformer. The two losses differ in supervision across positions, so this is documented rather than hidden behind a claim of identical training.

Linear and nonlinear autonomous dynamics

DMD solves a least-squares fit from consecutive training states. A linear propagator can be rolled forward without future observations. The learned models instead transform a history window, after which the oldest state is removed and the prediction appended.

$$K = \arg \min_K \|Z_{past} K - Z_{next}\|_F^2$$

$$\hat{Z}_{t+1} = F_{\theta}(\hat{Z}_{t-h+1}, \dots, \hat{Z}_t)$$

For a locally Lipschitz transition, a useful schematic error bound is next error no greater than L times current error plus local model defect. Iterating that bound explains why small defects can accumulate or amplify. It is not a rigorous bound for the trained network unless L and the region of validity have been established.

Checkpoint selection uses the autonomous validation trajectory, not just teacher-forced loss. An unstable candidate is assigned infinite validation loss and rejected; it must not terminate the complete sweep or cause successful seeds to disappear from the evidence.

The POD floor can dominate the field score

The decoder predicts an affine rank-eight subspace. Report representation error and in-subspace dynamics error separately. The retained evidence includes a numerical check of the Pythagorean residual, which should be near zero for in-subspace predictions with this uniformly weighted grid.

$$E_{field}^2 = E_{representation}^2 + E_{dynamics}^2$$

For example, a 5.45% representation floor and 1% dynamics error combine to about 5.54% field error. Halving the dynamics error would only lower field error to about 5.47%. Reporting only the latter pair of numbers can make a substantial dynamics improvement appear negligible. Conversely, a model already near the floor should not be advertised as accurately resolving structures absent from its decoder.

Persistence is a special control: its last observed full field need not lie in the fitted affine subspace. For that row the simple orthogonal decomposition need not close. The verifier therefore checks the identity for the in-subspace models and labels the exception instead of forcing every control into a convenient formula.

Horizon, phase and amplitude

A single aggregate norm hides when the forecast drifts. Plot per-frame error against steps since initialization, including the full validation-to-evaluation transition. Keep the POD floor visible. When showing additional observed initialization times, explicitly label them as another protocol; they are not additional independent trials of the original uninterrupted rollout.

The old 71-frame FFT window has coarse frequency resolution, approximately reciprocal to its observation duration. Identical peak bins do not show identical frequencies. The new diagnostic fits a constant plus sine and cosine over the complete 121-frame rollout, searching continuously within a declared physically relevant interval.

$$y(t) = c + a\sin(2\pi f(t - t_0)) + b\cos(2\pi f(t - t_0))$$

Here t_0 is the first rollout sample, so phase is measured relative to that origin. Constant signals have no identifiable frequency or phase. R-squared and residual MSE describe the adequacy of the sinusoidal approximation; neither validates lift-derived Strouhal. Amplitude is the square root of a squared plus b squared. Phase is $\text{atan2}(b,a)$. Report fit residual as well as frequency: a low residual supports the single-frequency approximation, while a large one signals that a more complex signal model is needed. A local vorticity probe frequency is not a force-derived Strouhal number.

A numerical phase example

If a prediction oscillates at 0.187 and the reference at 0.185 in nondimensional frequency, their frequency difference is only about 1.08%. Over 20 nondimensional time units, however, accumulated phase discrepancy is 2π times 0.002 times 20, approximately 0.251 radians. Amplitude mismatch and phase mismatch can therefore generate a noticeable field error even when both signals occupy the same FFT bin.

$$\Delta\phi(T) = 2\pi(\hat{f} - f)T$$

Initial phase error should be wrapped to a principal interval before comparison. Frequency-driven phase drift over the horizon should be reported separately, because wrapping the final accumulated drift would hide completed phase turns on a sufficiently long rollout.

Laboratory and assessment

The notebook independently reconstructs stored trajectories, recomputes per-frame errors, shows the matched MLP beside the Transformer and DMD, and checks the error decomposition. Four coding tasks implement history update, decomposition, a continuous-frequency diagnostic and an autonomous rollout audit with both per-step and global errors. The written interpretation keeps initial phase and accumulated phase drift separate.

Submit a horizon plot, the model-comparison table, a probe-fit table and one bounded conclusion. Explain whether an apparent advantage comes from dynamics or representation. Do not claim statistical significance from three training seeds or treat nearby errors as proof of equivalence; report the range and the limited case coverage.

Reading

Schmid, Dynamic mode decomposition of numerical and experimental data (2010), <https://doi.org/10.1017/S0022112010001217>, provides the primary DMD reference. Compare the existing Week 7 modal experiment and the new matched-information results before making an architecture claim. The causal training mechanism is derived in Week 18; this week evaluates its deployment behavior.

Printable laboratory worksheet

Task 1: Autonomous state update

Roll a history window forward by dropping its oldest token and appending the prediction. Do not read a future observed state.

Interface contract: `history:[context,features]` and `next_state:[features]` are NumPy arrays. Return a new `[context,features]` array in chronological order without changing the inputs.

Task 2: Separate representation and dynamics

Compute the squared total error, representation error and in-subspace error for an orthogonal projector.

Interface contract: `pred,truth,oracle` have identical shapes; `truth` is nonzero, `oracle` is an orthogonal affine projection and `pred` is in that affine span. Return a tuple of three scalar squared relative errors (`total,floor,dynamic`), each divided by `sum(truth2)`.

Task 3: Frequency-fit diagnostic

Measure relative frequency error using the continuous fitted frequency, not the closest FFT bin.

Interface contract: `t,y` are one-dimensional equal-length NumPy arrays, and `fref>0`. Return a scalar `abs(fitted_frequency/fref-1)`, using `lab.spectral_fit`. Only an oscillatory signal with an identifiable fit is supplied.

Task 4: Implement an autonomous rollout audit

Roll a history forward using only a supplied next-state function. Return predictions, per-step relative errors and one global relative error. Truth is used only after generating the trajectory. Validate dimensions and reject zero reference norms. Compare a stable and unstable predictor, then repeat from a different observed initialization without resetting at the evaluation boundary.

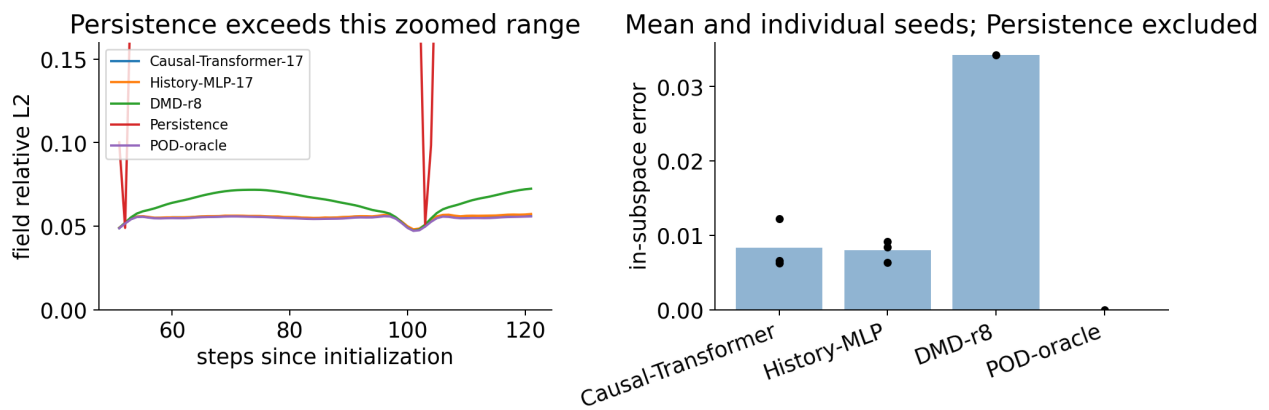
Interface contract: `predict_next` accepts a copy of `history:[context,features]` and returns one `[features]` vector. `truth:[steps,features]` sets the horizon and is used only for scoring after prediction. Return dictionary `predictions ([steps,features] array)`, `per_step_error (length-steps array)`, `field_error (float GLOBAL Frobenius relative L2, not the mean of per-step ratios)`. Invalid dimensions/state width, nonfinite predictions or any zero per-step truth norm must raise `ValueError`.

For the integrated fourth task, compare at least two controlled settings and explain a result that the implementation alone cannot justify. Include the requested plot or table and retain unsuccessful outcomes.

Before submitting

Restart the kernel and run the worked examples and completed tasks. Confirm that all four coding checks pass and that no retained data or results file changed. Include the requested interpretation, identify every quantity fitted from training data, and separate test observations from decisions made using validation. Record unresolved budget or representation limits rather than removing inconvenient runs.

Executed evidence and interpretation



This figure is generated from the retained protocol. Use the notebook to inspect all seeds and recompute the plotted quantities. The earlier lecture sections specify what the diagnostic can and cannot establish.